Student Name: _____

Student Email: _____



Homework 2 Quiz (CS 7150)
Fall 2026

Instructions

- Write your name and your Northeastern email address on every page.
- You have 20 minutes. Write your answers in the spaces directly after each question.
- For worked questions, draw a single BOX around your final answer, so that graders can tell the difference between your answer and your scratch work.
- Closed book: no notes, books, computers, calculators, or other resources, and no communication with anyone other than the proctor.
- You may not share or discuss these questions with any student who has not yet taken the quiz.

I attest that I have read and will follow the closed-book rules above.

Student signature: _____ Date: _____

1. (6 points) True or false, circle one.

   Consider two losses built from the same differentiable function f_θ, each optimized from the same starting point with the same learning rate:

   $$\mathcal{L}_1 = 10\, f_\theta(x, y)$$
   $$\mathcal{L}_2 = 0.1\, f_\theta(x, y)$$

   (a) Plain gradient descent takes a smaller first step on $\mathcal{L}_2$ than on $\mathcal{L}_1$. True False

   Solution: True. The step is $-\eta \nabla \mathcal{L}$, and $\nabla \mathcal{L}_2$ is 100 times smaller than $\nabla \mathcal{L}_1$, so the step is too. For gradient descent, rescaling the loss is the same as rescaling the learning rate, so a good learning rate depends on the scale of the loss.

   (b) ADAM takes a smaller first step on $\mathcal{L}_2$ than on $\mathcal{L}_1$. True False

   Solution: False. ADAM steps by $\eta\, m/\sqrt{v}$. Scaling every gradient by c scales m by c and $\sqrt{v}$ by c, so the ratio is unchanged and the two runs are identical at every step (ignoring ϵ). ADAM's η is roughly a step size in parameter units, whatever the scale of the loss.

   (c) With plain gradient descent, adding $\frac{\lambda}{2}\|\theta\|^2$ to the loss (L2 regularization) is exactly the same as shrinking every weight by $(1 - \eta\lambda)$ each step. True False

Solution: True. The extra gradient is $\lambda\theta$, so the update is $\theta \leftarrow \theta - \eta(\nabla\mathcal{L} + \lambda\theta) = (1 - \eta\lambda)\,\theta - \eta\nabla\mathcal{L}$.

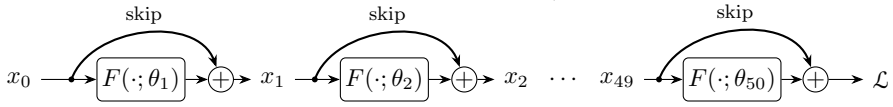
(d) With ADAM, adding $\frac{\lambda}{2}\|\theta\|^2$ to the loss is exactly the same as shrinking every weight by $(1 - \eta\lambda)$ each step. True False

Solution: False. The $\lambda\theta$ term goes into g, so it is averaged into m and divided by $\sqrt{v}$ like the rest of the gradient: weights with large gradients get almost no decay, and the decay no longer acts as a steady shrink. Applying the shrink separately, outside the gradient, is AdamW (decoupled weight decay).

(e) A deep MLP trains well with 64 units per hidden layer, initialized $w_{ij} \sim \mathcal{N}(0, \sigma^2)$. To widen every hidden layer to 1024 units, both Xavier Glorot and Kaiming He would recommend we initialize the hidden layers with standard deviation 4σ instead of σ. True False

Solution: False: it should be $\sigma/4$. Each unit sums n inputs, so the variance of its pre-activation is about $n\,\sigma^2$ times the variance of an input. Keeping that fixed when n grows 16-fold means dividing σ^2 by 16, i.e. σ by 4. This is why Xavier and He initialization scale the standard deviation as $1/\sqrt{n_{\text{in}}}$. Using 4σ would make each layer's signal 16 times larger than before, so a deep network would saturate or explode.

(f) Suppose in the 50-block residual network below, we set every weight θ_k to zero, so every F outputs 0. Then the gradient reaching the input, $\partial\mathcal{L}/\partial x_0$, will be zero. True False



Solution: False. Each block computes $x_k = x_{k-1} + F(x_{k-1};\theta_k)$, so $\partial x_k/\partial x_{k-1} = 1 + \partial F/\partial x_{k-1} = 1 + 0 = 1$, and the gradient $\partial\mathcal{L}/\partial x_{50}$ arrives at x_0 unchanged after passing through all 50 skip connections. In a plain 50-layer network with zero weights the gradient would be multiplied by zero at the first layer it met. Because the skip connection always carries the gradient, a residual block starts out as the identity, and adding depth does not make training harder.

For discussion: the gradient reaching the *activations* is intact, but if *every* weight in a branch is zero, the branch's own weights get zero gradient too (each layer's gradient is multiplied by the zero weights after it, or by the zero activations before it), so the branches never start learning. This is why in practice only the *last* layer of each branch is initialized to zero: the block still starts as the identity, and the earlier layers' nonzero activations let the last layer's weights receive a gradient.

2. As a reminder, here is the ADAM algorithm:

$$m_t^* = \beta_1 m_{t-1}^* + (1 - \beta_1)\, g_t$$
$$v_t^* = \beta_2 v_{t-1}^* + (1 - \beta_2)\, g_t^2$$
$$m_t = \frac{m_t^*}{1 - \beta_1^t}$$
$$v_t = \frac{v_t^*}{1 - \beta_2^t}$$
$$\Delta x_t = -\eta\, \frac{m_t}{\sqrt{v_t} + \epsilon}$$

In this problem, we will pick these EMA constants and learning rate and no epsilon:

$$\beta_1 = 0.5 \qquad \beta_2 = 0.5 \qquad \eta = 0.1 \qquad \epsilon = 0$$

and initialize:

$$m_0^* = 0 \qquad v_0^* = 0$$

Student Name: _____ Student Email: _____

Calculate two steps of ADAM. The first two gradients are $g_1 = -1$ and $g_2 = +1$.

(a) (5 points) Fill in the five empty boxes. Fractions are fine. Some values have been done.

t	g_t	m_t^*	v_t^*	m_t	v_t	Δx_t
1	-1	$-\frac{1}{2}$	$\frac{1}{2}$			0.1
2	$+1$	$+\frac{1}{4}$				$-\frac{1}{30}$

Solution:

t	g_t	m_t^*	v_t^*	m_t	v_t	Δx_t
1	-1	$-\frac{1}{2}$	$\frac{1}{2}$	$-\frac{1}{2}/\frac{1}{2} = \mathbf{-1}$	$\frac{1}{2}/\frac{1}{2} = \mathbf{1}$	0.1
2	$+1$	$+\frac{1}{4}$	$\frac{1}{4} + \frac{1}{2} = \mathbf{\frac{3}{4}}$	$\frac{1}{4}/\frac{3}{4} = \mathbf{\frac{1}{3}}$	$\frac{3}{4}/\frac{3}{4} = \mathbf{1}$	$-\frac{1}{30}$

The corrections are $1 - 0.5 = \frac{1}{2}$ at $t = 1$ and $1 - 0.5^2 = \frac{3}{4}$ at $t = 2$. Since $g_t^2 = 1$ at both steps, $v_t = 1$ throughout: only m feels the sign flip, so the second step is a third the size of the first and goes the other way. The given Δx values let a student check the work: $-0.1 \cdot m_t/\sqrt{v_t}$ should reproduce them.

Grading: 5 points, one per box (m_1, v_1, v_2^*, m_2, v_2). Carry an early arithmetic slip forward rather than penalizing it twice. Common errors: leaving out the bias correction ($m_1 = -\frac{1}{2}$, $v_1 = \frac{1}{2}$), forgetting the $(1-\beta)$ factor in v_2^*, or dividing by the $t = 1$ correction $\frac{1}{2}$ instead of $\frac{3}{4}$ at $t = 2$.

(b) (2 points, extra credit) If we changed g_2, ADAM could take a bigger second step. What value of g_2 would result in $|m_2| = \sqrt{v_2}$? (Hint: when is a mean as large as an RMS?)

$g_2 =$ _____

Solution: $\mathbf{g_2 = -1}$, the same as g_1.

In general $m_2^* = -\frac{1}{4} + \frac{1}{2}g_2$ and $v_2^* = \frac{1}{4} + \frac{1}{2}g_2^2$, so after dividing by $\frac{3}{4}$, $m_2 = \frac{2g_2 - 1}{3}$ and $v_2 = \frac{1+2g_2^2}{3}$. Setting $m_2^2 = v_2$ gives $(2g_2 - 1)^2 = 3(1 + 2g_2^2)$, i.e. $2g_2^2 + 4g_2 + 2 = 0$, i.e. $(g_2 + 1)^2 = 0$. Then $m_2 = -1$, $v_2 = 1$, and the step is $\Delta x_2 = +0.1$: three times the $\frac{1}{30}$ above, in the same direction as the first step, and the largest step ADAM ever takes.

The principle: m is a weighted mean of the gradients and $\sqrt{v}$ is their RMS with the same weights, and a mean is never larger than the RMS, with equality only when every gradient is the same. So $|m/\sqrt{v}| \leq 1$, and ADAM takes its full step η only when the gradient is perfectly consistent; a gradient that flips sign gets a much smaller step.

Grading: 2 points for $g_2 = -1$. Give 1 point for an answer that sets up $m_2^2 = v_2$ correctly (in the margin) but slips in the algebra.

3. Two broken PyTorch optimizers. Both of these `step()` methods are meant to do plain gradient descent on the parameters the optimizer was given.

Student Name: _____ Student Email: _____

Version 1

```
def step(self):
    with torch.no_grad():
        for x in self.parameters:
            x = x - self.lr * x.grad
```

Version 2

```
def step(self):
    for x in self.parameters:
        x -= self.lr * x.grad
```

(a) (2 points) Which version raises "`RuntimeError: a leaf Variable that requires grad is being used in an in-place operation`"? Circle one: Version 1 Version 2

Solution: Version 2. Each parameter is a leaf tensor with `requires_grad=True`, and autograd refuses to record an in-place change to it. Updates belong inside `torch.no_grad()`.

(b) (2 points) Which version runs without error, but the loss printed in the training loop never changes? Circle one: Version 1 Version 2

Solution: Version 1. `x - self.lr * x.grad` builds a *new* tensor, and `x = ...` only rebinds the loop variable to it. The parameter tensor that the model and `self.parameters` hold is never written to, so the weights never move. (The `no_grad` is not the problem: without it, the rebinding still changes nothing.)

(c) (3 points) Write a `step()` that has neither bug.

```
def step(self):
```

Solution:

```
def step(self):
    with torch.no_grad():
        for x in self.parameters:
            x -= self.lr * x.grad
```

Version 1's `no_grad` plus Version 2's in-place update. Also accept `x.sub_(self.lr * x.grad)` or `x.data -= self.lr * x.grad`.

Grading: 3 points: 1 for the loop over `self.parameters` using `self.lr` and `x.grad`, 1 for an in-place update (`-=` or `sub_`), and 1 for doing it inside `torch.no_grad()` (or through `.data`).